

Exam Prep – C Funktionen

Contents

1 Bitfield (bitfield.c)	2
1.1 set_bit	2
1.2 clear_bit	2
1.3 toggle_bit	2
1.4 get_bit	2
1.5 count_ones	2
1.6 pack_rgb / unpack_rgb	2
2 Warehouse / Inventory (inventory.c)	3
2.1 warehouse_create / warehouse_destroy	3
2.2 warehouse_add_product	3
2.3 warehouse_remove_by_id	3
2.4 warehouse_find_by_name / warehouse_total_value	3
2.5 warehouse_sort_by_price	4
3 Verkettete Liste (linkedlist.c)	4
3.1 slist_create / slist_destroy	4
3.2 slist_push_front / slist_push_back	5
3.3 slist_pop_front	5
3.4 slist_remove_at	5
3.5 slist_insertion_sort	6
3.6 slist_reverse / slist_to_array	6
4 Rekursion (recursion.c)	6
4.1 fast_power	6
4.2 gcd	7
4.3 hanoi	7
4.4 count_paths / count_paths_memo	7
5 Encoding (encode.c)	7
5.1 caesar_encode / caesar_decode	7
5.2 xor_encode	8
5.3 is_rotation	8
5.4 rle_encode / rle_decode	8
6 Exam Prep Funktionen (exam_prep.c)	9
6.1 compress_rle	9
6.2 find_lowest_stock / apply_discount	9
6.3 create_matrix / transpose_matrix / free_matrix	10
6.4 list_append / list_filter_even / list_free	10
7 Exam-Prep2	11
7.1 Strings	11
7.2 Structs	12
7.3 Stack	13
7.4 LinkedList	14

1 Bitfield (bitfield.c)

1.1 set_bit

```
1 uint32_t set_bit(uint32_t val, int pos)
2 {
3     return val | (1U << pos);
4 }
```

1.2 clear_bit

```
1 uint32_t clear_bit(uint32_t val, int pos)
2 {
3     return val & ~(1U << pos);
4 }
```

1.3 toggle_bit

```
1 uint32_t toggle_bit(uint32_t val, int pos)
2 {
3     return val ^ (1U << pos);
4 }
```

1.4 get_bit

```
1 int get_bit(uint32_t val, int pos)
2 {
3     return (val >> pos) & 1U;
4 }
```

OR setzt, AND+NOT loescht, XOR kippt, Shift+Mask liest ein Bit.

1.5 count_ones

```
1 int count_ones(uint32_t val)
2 {
3     int count = 0;
4     while (val > 0) { count += val & 1; val >>= 1; }
5     return count;
6 }
```

Zaehlt gesetzte Bits (Hamming-Gewicht). Prueft LSB, schiebt rechts.

1.6 pack_rgb / unpack_rgb

```
1 uint32_t pack_rgb(uint8_t r, uint8_t g, uint8_t b)
2 {
3     return ((uint32_t)r << 16) | ((uint32_t)g << 8) | (uint32_t)b;
4 }
5
6 void unpack_rgb(uint32_t packed, uint8_t *r, uint8_t *g, uint8_t *b)
7 {
8     if (r) *r = (packed >> 16) & 0xFF;
9     if (g) *g = (packed >> 8) & 0xFF;
10    if (b) *b = packed & 0xFF;
11 }
```

Format: 0x00RRGGBB. R in Bits 23-16, G in 15-8, B in 7-0.

2 Warehouse / Inventory (inventory.c)

2.1 warehouse_create / warehouse_destroy

```
1 Warehouse *warehouse_create(void)
2 {
3     Warehouse *w = malloc(sizeof(Warehouse));
4     if (!w) return NULL;
5     w->products = NULL; w->count = 0; w->capacity = 4; w->next_id = 1;
6     return w;
7 }
8
9 void warehouse_destroy(Warehouse *w)
10 {
11     if (!w) return;
12     free(w->products);
13     free(w);
14 }
```

Create initialisiert alle Felder. Destroy gibt erst Produkt-Array, dann Struktur frei.

2.2 warehouse_add_product

```
1 int warehouse_add_product(Warehouse *w, const char *name,
2                           double price, int quantity)
3 {
4     if (!w || !name) return -1;
5     if (w->count >= w->capacity) {
6         int new_cap = w->capacity == 0 ? 4 : w->capacity * 2;
7         Product *tmp = realloc(w->products, new_cap * sizeof(Product));
8         if (!tmp) return -1;
9         w->products = tmp; w->capacity = new_cap;
10    }
11    w->products[w->count].id = w->next_id++;
12    w->products[w->count].price = price;
13    w->products[w->count].quantity = quantity;
14    strncpy(w->products[w->count].name, name, 63);
15    w->products[w->count].name[63] = '\0';
16    w->count++;
17    return 0;
18 }
```

Bei vollem Array: realloc auf doppelte Kapazitaet. strncpy + manueller Null-Terminator gegen Buffer-Overflow.

2.3 warehouse_remove_by_id

```
1 int warehouse_remove_by_id(Warehouse *w, int id)
2 {
3     if (!w) return -1;
4     for (int i = 0; i < w->count; i++) {
5         if (w->products[i].id == id) {
6             for (int j = i; j < w->count - 1; j++)
7                 w->products[j] = w->products[j + 1];
8             w->count--;
9             return 0;
10        }
11    }
12    return -1;
13 }
```

Lineare Suche, dann Linksverschiebung aller nachfolgenden Elemente.

2.4 warehouse_find_by_name / warehouse_total_value

```

1 Product *warehouse_find_by_name(Warehouse *w, const char *query)
2 {
3     if (!w || !query) return NULL;
4     for (int i = 0; i < w->count; i++)
5         if (strcmp(w->products[i].name, query) == 0)
6             return &w->products[i];
7     return NULL;
8 }
9
10 double warehouse_total_value(const Warehouse *w)
11 {
12     if (!w) return 0.0;
13     double total = 0;
14     for (int i = 0; i < w->count; i++)
15         total += w->products[i].price * w->products[i].quantity;
16     return total;
17 }

```

Find gibt Zeiger ins Array zurueck (kein Kopieren). Total Value: Summe von Preis x Menge.

2.5 warehouse_sort_by_price

```

1 void warehouse_sort_by_price(Warehouse *w)
2 {
3     if (!w || w->count <= 1) return;
4     for (int i = 0; i < w->count - 1; i++) {
5         for (int j = 0; j < w->count - i - 1; j++) {
6             if (w->products[j].price > w->products[j+1].price ||
7                 (w->products[j].price == w->products[j+1].price &&
8                  w->products[j].id > w->products[j+1].id)) {
9                 Product tmp = w->products[j];
10                w->products[j] = w->products[j+1];
11                w->products[j+1] = tmp;
12            }
13        }
14    }
15 }

```

Bubble Sort nach Preis. Bei Gleichstand nach ID sortiert (deterministisch).

3 Verkettete Liste (linkedlist.c)

3.1 slist_create / slist_destroy

```

1 SList *slist_create(void)
2 {
3     SList *l = malloc(sizeof(SList));
4     if (!l) return NULL;
5     l->head = NULL; l->tail = NULL; l->size = 0;
6     return l;
7 }
8
9 void slist_destroy(SList *list)
10 {
11     if (!list) return;
12     SNode *cur = list->head;
13     while (cur) { SNode *nxt = cur->next; free(cur); cur = nxt; }
14     free(list);
15 }

```

Tail-Pointer fuer O(1) push_back. Destroy: next vor free sichern!

3.2 slist_push_front / slist_push_back

```
1 int slist_push_front(SList *list, int value)
2 {
3     if (!list) return -1;
4     SNode *n = malloc(sizeof(SNode));
5     if (!n) return -1;
6     n->value = value; n->next = list->head;
7     list->head = n;
8     if (list->size == 0) list->tail = n;
9     list->size++;
10    return 0;
11 }
12
13 int slist_push_back(SList *list, int value)
14 {
15     if (!list) return -1;
16     SNode *n = malloc(sizeof(SNode));
17     if (!n) return -1;
18     n->value = value; n->next = NULL;
19     if (list->size == 0) { list->head = n; list->tail = n; }
20     else { list->tail->next = n; list->tail = n; }
21     list->size++;
22     return 0;
23 }
```

Beide $O(1)$. push_front: neuer Knoten zeigt auf alten Head. push_back: Tail-Pointer ermöglicht direktes Anhaengen.

3.3 slist_pop_front

```
1 int slist_pop_front(SList *list, int *err)
2 {
3     if (!list || list->size == 0) { if (err) *err = -1; return 0; }
4     SNode *rem = list->head;
5     int val = rem->value;
6     list->head = rem->next;
7     if (--list->size == 0) list->tail = NULL;
8     free(rem);
9     if (err) *err = 0;
10    return val;
11 }
```

Output-Parameter err, da C keinen zweiten Rueckgabewert hat. Leere Liste: 0 + err=-1.

3.4 slist_remove_at

```
1 int slist_remove_at(SList *list, int index)
2 {
3     if (!list || index < 0 || index >= list->size) return -1;
4     if (index == 0) {
5         SNode *n = list->head;
6         list->head = n->next;
7         if (--list->size == 0) list->tail = NULL;
8         free(n); return 0;
9     }
10    SNode *prev = list->head;
11    for (int i = 0; i < index - 1; i++) prev = prev->next;
12    SNode *n = prev->next;
13    prev->next = n->next;
14    if (n == list->tail) list->tail = prev;
15    list->size--;
16    free(n);
17    return 0;
18 }
```

Index 0 ist Sonderfall (kein Vorgaenger). Tail aktualisieren wenn letztes Element geloescht wird.

3.5 slist_insertion_sort

```
1 void slist_insertion_sort(SList *list)
2 {
3     if (!list || list->size <= 1) return;
4     SNode *sorted_head = NULL, *sorted_tail = NULL;
5     SNode *cur = list->head;
6     while (cur) {
7         SNode *nxt = cur->next;
8         if (!sorted_head || cur->value <= sorted_head->value) {
9             cur->next = sorted_head; sorted_head = cur;
10            if (!sorted_tail) sorted_tail = cur;
11        } else {
12            SNode *s = sorted_head;
13            while (s->next && s->next->value < cur->value) s = s->next;
14            cur->next = s->next; s->next = cur;
15            if (!cur->next) sorted_tail = cur;
16        }
17        cur = nxt;
18    }
19    list->head = sorted_head; list->tail = sorted_tail;
20 }
```

Kein neuer Speicher: Knoten direkt in sortierte Teilliste umgehaengt. $O(n^2)$.

3.6 slist_reverse / slist_to_array

```
1 void slist_reverse(SList *list)
2 {
3     if (!list || list->size <= 1) return;
4     SNode *prev = NULL, *cur = list->head, *nxt;
5     list->tail = list->head;
6     while (cur) { nxt = cur->next; cur->next = prev; prev = cur; cur = nxt; }
7     list->head = prev;
8 }
9
10 int *slist_to_array(const SList *list)
11 {
12     if (!list || list->size == 0) return NULL;
13     int *arr = malloc(list->size * sizeof(int));
14     if (!arr) return NULL;
15     SNode *cur = list->head;
16     for (int i = 0; i < list->size; i++, cur = cur->next) arr[i] = cur->value;
17     return arr;
18 }
```

Reverse: drei Zeiger, $O(n)$, $O(1)$ Speicher. to_array: Aufrufer muss freigeben!

4 Rekursion (recursion.c)

4.1 fast_power

```
1 long long fast_power(long long x, int n)
2 {
3     if (n == 0) return 1;
4     long long half = fast_power(x, n / 2);
5     return (n % 2 == 0) ? half * half : x * half * half;
6 }
```

Exponentiation by Squaring. $x^n = (x^{(n/2)})^2$ (gerade) bzw. $x * (x^{(n/2)})^2$ (ungerade). $O(\log n)$.

4.2 gcd

```
1 int gcd(int a, int b)
2 {
3     if (b == 0) return a;
4     return gcd(b, a % b);
5 }
```

Euklidischer Algorithmus: $\text{ggT}(a,b) = \text{ggT}(b, a \bmod b)$, Abbruch bei $b=0$.

4.3 hanoi

```
1 int hanoi(int n, char from, char to, char aux)
2 {
3     if (n == 0) return 0;
4     int moves = 0;
5     moves += hanoi(n-1, from, aux, to);
6     printf("Bewege Scheibe %d von %c nach %c\n", n, from, to);
7     moves++;
8     moves += hanoi(n-1, aux, to, from);
9     return moves;
10 }
```

$n-1$ Scheiben nach aux, Scheibe n direkt, $n-1$ Scheiben nach to. Gesamt: $2^n - 1$ Zuege.

4.4 count_paths / count_paths_memo

```
1 int count_paths(int rows, int cols)
2 {
3     if (rows == 1 || cols == 1) return 1;
4     return count_paths(rows-1, cols) + count_paths(rows, cols-1);
5 }
6
7 int count_paths_memo(int rows, int cols, int memo[][20])
8 {
9     if (rows == 1 || cols == 1) return 1;
10    if (memo && memo[rows-1][cols-1] != -1) return memo[rows-1][cols-1];
11    int r = count_paths_memo(rows-1, cols, memo)
12          + count_paths_memo(rows, cols-1, memo);
13    if (memo) memo[rows-1][cols-1] = r;
14    return r;
15 }
```

Pfade in Gitter (nur rechts/runter). Memo mit -1 initialisieren. Reduziert von exponentiell auf $O(\text{rows} \cdot \text{cols})$.

5 Encoding (encode.c)

5.1 caesar_encode / caesar_decode

```
1 char *caesar_encode(const char *text, int shift)
2 {
3     if (!text) return NULL;
4     size_t len = strlen(text);
5     char *res = malloc(len + 1);
6     if (!res) return NULL;
7     shift = ((shift % 26) + 26) % 26;
8     for (size_t i = 0; i < len; i++) {
9         char c = text[i];
10        if (c >= 'a' && c <= 'z') res[i] = 'a' + (c - 'a' + shift) % 26;
11        else if (c >= 'A' && c <= 'Z') res[i] = 'A' + (c - 'A' + shift) % 26;
12        else res[i] = c;
13    }
14    res[len] = '\0';
15    return res;
}
```

```

16 }
17
18 char *caesar_decode(const char *text, int shift)
19 {
20     int s = ((shift % 26) + 26) % 26;
21     return caesar_encode(text, (26 - s) % 26);
22 }

```

Shift um n Positionen, wrap-around. Decode ruft Encode mit Gegenshift auf. Aufrufer muss freigeben.

5.2 xor_encode

```

1 char *xor_encode(const char *text, uint8_t key)
2 {
3     if (!text) return NULL;
4     size_t len = strlen(text);
5     char *res = malloc(len + 1);
6     if (!res) return NULL;
7     for (size_t i = 0; i < len; i++) res[i] = text[i] ^ key;
8     res[len] = '\0';
9     return res;
10 }

```

Symmetrisch: doppeltes XOR mit gleichem Key ergibt Original.

5.3 is_rotation

```

1 int is_rotation(const char *a, const char *b)
2 {
3     if (!a || !b) return 0;
4     size_t la = strlen(a), lb = strlen(b);
5     if (la != lb) return 0;
6     if (la == 0) return 1;
7     char *concat = malloc(la * 2 + 1);
8     if (!concat) return 0;
9     strcpy(concat, a); strcat(concat, a);
10    int r = (strstr(concat, b) != NULL) ? 1 : 0;
11    free(concat);
12    return r;
13 }

```

Trick: jede Rotation von a ist Teilstring in aa.

5.4 rle_encode / rle_decode

```

1 char *rle_encode(const char *text)    // "aaabbbc" -> "3a2b1c"
2 {
3     if (!text) return NULL;
4     size_t len = strlen(text);
5     char *res = malloc(len * 12 + 1);
6     if (!res) return NULL;
7     int count = 1; char cur = text[0]; size_t pos = 0;
8     for (size_t i = 1; i <= len; i++) {
9         if (i < len && text[i] == cur) { count++; }
10        else {
11            pos += sprintf(res + pos, "%d%c", count, cur);
12            if (i < len) { cur = text[i]; count = 1; }
13        }
14    }
15    char *s = realloc(res, pos + 1);
16    return s ? s : res;
17 }
18
19 char *rle_decode(const char *enc)    // "3a2b1c" -> "aaabbbc"

```



```

20 {
21     if (!enc || !enc[0]) { char *r = malloc(1); if(r) *r='\0'; return r; }
22     size_t total = 0, i = 0, len = strlen(enc);
23     while (i < len) {
24         int c = 0;
25         while (i < len && isdigit((unsigned char)enc[i]))
26             c = c * 10 + (enc[i++] - '0');
27         if (!c || i >= len) return NULL;
28         total += c; i++;
29     }
30     char *res = malloc(total + 1); if (!res) return NULL;
31     i = 0; size_t pos = 0;
32     while (i < len) {
33         int c = 0;
34         while (i < len && isdigit((unsigned char)enc[i]))
35             c = c * 10 + (enc[i++] - '0');
36         char ch = enc[i++];
37         while (c-->0) res[pos++] = ch;
38     }
39     res[total] = '\0';
40     return res;
41 }

```

Encode: grosszuegige Allokation, dann realloc. Decode: zwei Paesse – erst Laenge, dann aufbauen.

6 Exam Prep Funktionen (exam_prep.c)

6.1 compress_rle

```

1 void compress_rle(const char *src, char *dest) // "aaabbbc" -> "a3b2c1"
2 {
3     if (src[0] == '\0') { dest[0] = '\0'; return; }
4     char cur = src[0]; int count = 1, pos = 0;
5     for (int i = 1; src[i] != '\0'; i++) {
6         if (cur == src[i]) count++;
7         else {
8             pos += sprintf(dest + pos, "%c%d", cur, count);
9             cur = src[i]; count = 1;
10        }
11    }
12    sprintf(dest + pos, "%c%d", cur, count);
13 }

```

Wie rle_encode aber kein Heap: schreibt direkt in Aufrufer-Buffer. Format: Zeichen dann Zahl. Kein Laengencheck!

6.2 find_lowest_stock / apply_discount

```

1 Item *find_lowest_stock(Item *items, int length)
2 {
3     Item *lowest = &items[0];
4     for (int i = 1; i < length; i++)
5         if (items[i].quantity < lowest->quantity) lowest = &items[i];
6     return lowest;
7 }
8
9 void apply_discount(Item *items, int length,
10                    const char *category, float pct)
11 {
12     for (int i = 0; i < length; i++)
13         if (strcmp(items[i].category, category) == 0)
14             items[i].price *= (1.0f - pct / 100.0f);
15 }

```

find_lowest gibt Zeiger ins Array zurueck. apply_discount modifiziert Preise in-place.

6.3 create_matrix / transpose_matrix / free_matrix

```
1 int **create_matrix(int rows, int cols)
2 {
3     int **m = malloc(sizeof(int*) * rows);
4     if (!m) return NULL;
5     for (int i = 0; i < rows; i++) {
6         m[i] = malloc(sizeof(int) * cols);
7         if (!m[i]) {
8             for (int j = 0; j < i; j++) free(m[j]);
9             free(m); return NULL;
10        }
11    }
12    return m;
13 }
14
15 int **transpose_matrix(int **m, int rows, int cols)
16 {
17     int **t = create_matrix(cols, rows);
18     for (int i = 0; i < rows; i++)
19         for (int j = 0; j < cols; j++)
20             t[j][i] = m[i][j];
21     return t;
22 }
23
24 void free_matrix(int **m, int rows)
25 {
26     for (int i = 0; i < rows; i++) free(m[i]);
27     free(m);
28 }
```

Jede Zeile separat allokiert. Bei Fehler: Cleanup aller bisherigen Zeilen. Transpose: $[i][j] \rightarrow [j][i]$. free: erst Zeilen, dann Zeigerarray.

6.4 list_append / list_filter_even / list_free

```
1 void list_append(Node **head, int value)
2 {
3     Node *n = malloc(sizeof(Node));
4     if (!n) return;
5     n->data = value; n->next = NULL;
6     if (!*head) { *head = n; return; }
7     Node *cur = *head;
8     while (cur->next) cur = cur->next;
9     cur->next = n;
10 }
11
12 void list_filter_even(Node **head)
13 {
14     Node *cur = *head;
15     while (cur && cur->next) {
16         if (cur->next->data % 2 == 0) {
17             Node *del = cur->next;
18             cur->next = del->next;
19             free(del);
20         } else cur = cur->next;
21     }
22 }
23
24 void list_free(Node **head)
25 {
26     Node *cur = *head;
27     while (cur) { Node *nxt = cur->next; free(cur); cur = nxt; }
28     *head = NULL;
29 }
```

append: $O(n)$, kein Tail-Pointer. filter_even: loescht ueber Nachfolger (Head wird nicht geprueft!). list_free setzt Pointer auf NULL.

7 Exam-Prep2

7.1 Strings

```
1  int is_palindrome(const char *s)
2  {
3      if(s == NULL) return 0;
4
5      int len = strlen(s);
6
7      if (len == 0) return 1;
8
9      int left = 0;
10     int right = len - 1;
11
12     while(left < right)
13     {
14         if(tolower(s[left]) != tolower(s[right]))
15         {
16             return 0;
17         }
18
19         left++;
20         right--;
21     }
22
23     return 1;
24 }
25
26
27 int count_words(const char *s)
28 {
29     if(s == NULL) return 0;
30     int count = 0;
31     int i = 0;
32     int in_word = 0;
33
34     while (s[i] != '\0')
35     {
36         if (s[i] != ' ')
37         {
38             if (in_word == 0)
39             {
40                 count++;
41                 in_word = 1;
42             }
43         }
44         else
45         {
46             in_word = 0;
47         }
48         i++;
49     }
50
51     return count;
52 }
53
54 char *str_reverse(const char *s)
55 {
56     if (s == NULL) return NULL;
57
58     int len = strlen(s);
```

```

59     char *reversed = malloc(len + 1);
60
61     if (reversed == NULL) return NULL;
62
63     for (int i = 0; i < len; i++)
64     {
65         reversed[i] = s[len - 1 - i];
66     }
67
68     reversed[len] = '\0';
69
70     return reversed;
71 }
72
73 int str_replace_char(char *s, char old_char, char new_char)
74 {
75     if(s == NULL) return -1;
76
77     int len = strlen(s);
78     int count = 0;
79
80     //for(int i = 0; s[i] != '\0'; i++)    damit braucht man strlen in diesem fall
81     //nicht
82     for(int i = 0; i < len; i++)
83     {
84         if(s[i] == old_char)
85         {
86             s[i] = new_char;
87             count++;
88         }
89     }
90     return count;
91 }

```

7.2 Structs

```

1
2 double student_average(const Student *s)
3 {
4     if(s == NULL || s->grade_count == 0) return 0.0;
5
6     double all_grades = s->grades[0];
7
8     for(int i = 1; i < s->grade_count; i++)
9     {
10         all_grades += s->grades[i];
11     }
12
13     double average = all_grades / (double)s->grade_count;
14     return average;
15 }
16
17 Student *find_best_student(Student *students, int n)
18 {
19     if(n <= 0 || students == NULL) return NULL;
20
21     int best_index = 0;
22     float best_grades = student_average(&students[0]);
23
24     for (unsigned int i = 1; i < n; i++)
25     {
26         if(student_average(&students[i]) < best_grades)
27         {
28             best_grades = student_average(&students[i]);

```

```

29         best_index = i;
30     }
31
32 }
33
34 return &students[best_index];
35 }
36
37 void sort_students_by_average(Student *students, int n)
38 {
39     if(n <= 0 || students == NULL) return;
40
41     for (int i = 0; i < n - 1; i++)
42     {
43         for (int j = 0; j < n - i - 1; j++)
44         {
45             if(student_average(&students[j]) > student_average(&students[j + 1]))
46             {
47                 Student temp = students[j];
48                 students[j] = students[j + 1];
49                 students[j + 1] = temp;
50             }
51         }
52     }
53 }
54
55 int student_passed_all(const Student *s)
56 {
57     if(s == NULL || s->grade_count == 0) return 0;
58
59     for(int i = 0; i < s->grade_count; i++)
60     {
61         if(s->grades[i] > 4.0)
62         {
63             return 0;
64         }
65     }
66
67     return 1;
68 }

```

7.3 Stack

```

1
2 Stack *stack_create(void)
3 {
4     Stack *s = malloc(sizeof(Stack));
5     if (s == NULL) return NULL;
6
7     s->data = malloc(4 * sizeof(int));
8     if (s->data == NULL)
9     {
10         free(s);
11         return NULL;
12     }
13
14     s->size = 0;
15     s->capacity = 4;
16
17     return s;
18 }
19
20 void stack_destroy(Stack *s)
21 {
22     if(s == NULL) return;

```

```

23
24     if (s->data != NULL)
25     {
26         free(s->data);
27     }
28
29     free(s);
30 }
31
32 int stack_push(Stack *s, int value)
33 {
34     if (s == NULL) return -1;
35
36     if (s->size == s->capacity)
37     {
38         int new_capacity = (s->capacity == 0) ? 4 : s->capacity * 2;
39
40         int *new_data = realloc(s->data, new_capacity * sizeof(int));
41
42         if (new_data == NULL) return -1;
43
44         s->data = new_data;
45         s->capacity = new_capacity;
46     }
47
48     s->data[s->size] = value;
49     s->size++;
50
51     return 0;
52 }
53
54 int stack_pop(Stack *s, int *err)
55 {
56     if (s == NULL || s->size == 0)
57     {
58         if (err != NULL) *err = -1;
59         return 0;
60     }
61
62     if (err != NULL) *err = 0;
63
64     s->size--;
65     return s->data[s->size];
66 }
67
68 int stack_peek(const Stack *s, int *err)
69 {
70     if (s == NULL || s->size == 0)
71     {
72         if (err != NULL) *err = -1;
73         return 0;
74     }
75     if (err != NULL) *err = 0;
76
77     return s->data[s->size - 1];
78 }
79
80 int stack_is_empty(const Stack *s)
81 {
82     if (s == NULL || s->size == 0) return 1;
83
84     return 0;
85 }

```

7.4 LinkedList

```

1 void list_append(Node **head, int value)
2 {
3     if (head == NULL) return;
4
5     Node *new_node = malloc(sizeof(Node));
6     if (new_node == NULL) return;
7
8     new_node->data = value;
9     new_node->next = NULL;
10
11     if (*head == NULL)
12     {
13         *head = new_node;
14         return;
15     }
16
17     Node *curr = *head;
18     while (curr->next != NULL)
19     {
20         curr = curr->next;
21     }
22     curr->next = new_node;
23 }
24
25 void list_free(Node **head)
26 {
27     if (head == NULL) return;
28
29     Node *curr = *head;
30     while (curr != NULL)
31     {
32         Node *next = curr->next;
33         free(curr);
34         curr = next;
35     }
36     *head = NULL;
37 }
38
39 int list_sum(const Node *head)
40 {
41     int sum = 0;
42     const Node *curr = head;
43
44     while (curr != NULL)
45     {
46         sum += curr->data;
47         curr = curr->next;
48     }
49
50     return sum;
51 }
52
53 void list_remove_duplicates(Node **head)
54 {
55     if (head == NULL || *head == NULL) return;
56
57     Node *curr = *head;
58
59     while (curr != NULL)
60     {
61         Node *runner = curr;
62
63         while (runner->next != NULL)
64         {
65

```

```

66         if (runner->next->data == curr->data)
67         {
68             Node *duplicate = runner->next;
69             runner->next = runner->next->next;
70             free(duplicate);
71         }
72         else
73         {
74             runner = runner->next;
75         }
76     }
77     curr = curr->next;
78 }
79 }
80
81 int list_is_sorted(const Node *head)
82 {
83     if (head == NULL || head->next == NULL) return 1;
84
85     const Node *curr = head;
86
87     while (curr->next != NULL)
88     {
89         if (curr->data > curr->next->data) return 0;
90         curr = curr->next;
91     }
92
93     return 1;
94 }
95
96 void list_insert_sorted(Node **head, int value)
97 {
98     if (head == NULL) return;
99
100     Node *new_node = malloc(sizeof(Node));
101     if (new_node == NULL) return;
102
103     new_node->data = value;
104
105     if (*head == NULL || (*head)->data >= value)
106     {
107         new_node->next = *head;
108         *head = new_node;
109         return;
110     }
111
112     Node *curr = *head;
113     while (curr->next != NULL && curr->next->data < value)
114     {
115         curr = curr->next;
116     }
117
118     new_node->next = curr->next;
119     curr->next = new_node;
120 }

```